Customer Journey Map

Script-Controlled ACL – Institution Record Access

Date	1 October 2026
Team ID	SWTID-2026-8643
Project Name	Script-Controlled ACL – Control Record Access Based on Field Value (ServiceNow)
Persona	ServiceNow administrator who secures the Institution Details table, and the EEE branch user whose access is tested
Basis	Walkthrough of the lab steps and its verification screens

SCENARIO **Configuring, using and validating script-controlled ACLs for Institution Details records**	**Entice** How does someone initially become aware of this process?	**Enter** What do people experience as they begin the process?	**Engage** In the core moments in the process, what happens?	**Exit** What do people typically experience as the process finishes?	**Extend** What happens after the experience is over?
Steps What does the person (or group) typically experience?	• Administrator notices that Institution Details records are visible to all users • Decides that records should be restricted according to Branch • Opens the Script-Controlled ACL lab	• Creates EEE User • Creates roles bb1, bb2, bb3, bb4 and assigns them to EEE User • Creates the u_institution_details table with its seven fields • Adds sample records for ECE, EEE and CSE	• Elevates the security_admin role • Creates the Read ACL (bb1, Branch is EEE, script) • Creates the Create ACL (bb2) • Creates the Write ACL (bb3) • Creates the Delete ACL (bb4)	• Impersonates EEE User and opens the list • Impersonates a user without roles • Impersonates Admin and opens the list • Tests New, inline edit and Delete as roles are added	• Records remain protected after the lab • The same approach can be reused for other branches or tables • ACLs and role assignments are reviewed and maintained
Interactions What interactions do they have at each step? People, places, things.	• People: lab instructor, team members • Places: ServiceNow instance, logged in as admin • Things: lab document, Application Navigator	• Places: Users, Roles and Tables modules • Things: user form, role form, table columns form, Institution Details list	• Places: System Security → Access Control (ACL) • Things: Elevate role dialog, ACL form, Requires role list, Data Condition, Script field	• Places: Impersonate user menu, u_institution_details list • Things: New button, Actions on selected rows, delete confirmation	• Places: forms, lists and Service Portal • Things: Access Control list, role assignments
Goals & motivations At each step, what is a person's primary goal or motivation? ("Help me..." or "Help me avoid...")	• Help me keep student records protected by branch • Help me understand what ACLs can do	• Help me prepare test data efficiently • Help me avoid mixing up the bb1 – bb4 roles	• Help me provide the correct access for each operation • Help me avoid locking admins out	• Help me verify that each role sees only what it should	• Help me reuse this access pattern for other data • Help me keep access rules simple to maintain

SCENARIO **Configuring, using and validating script-controlled ACLs for Institution Details records**	**Entice** How does someone initially become aware of this process?	**Enter** What do people experience as they begin the process?	**Engage** In the core moments in the process, what happens?	**Exit** What do people typically experience as the process finishes?	**Extend** What happens after the experience is over?
Positive moments What steps does a typical person find enjoyable, productive, fun, motivating, delightful, or exciting?	• The scenario is clear: EEE users see only EEE records, admins keep full access	• Searching in the Application Navigator makes each module easy to find • Records from three branches make the later test easy to see	• Every ACL follows the same pattern • The script gives full control: admin override, bb1 check, deny for everyone else	• Instant, visible result: 2 EEE records for the EEE user versus all 6 for Admin • "No records to display" proves a user without the role is blocked	• The pattern protects data across forms, lists and Service Portal
Negative moments What steps does a typical person find frustrating, confusing, angering, costly, or time-consuming?	• Many new ideas at once: roles, ACLs, scripts, impersonation • Elevating security_admin is easy to miss until an ACL will not save	• Four similarly named roles are easy to confuse • Seven fields must be added one by one	• Role, data condition and script all have to pass, which is easy to misread • Forgetting to elevate the role blocks saving the ACL	• Switching between impersonated users is repetitive • Easy to forget which roles the test user holds when testing create, write and delete	• One ACL per branch could multiply quickly • Role and branch are hard-coded in the script
Areas of opportunity How might we make each step better? What ideas do we have? What have others suggested?	• How might we give a one-page overview of the four ACL operations up front? • Add a prerequisites checklist (security_admin elevation)	• Name roles by purpose, e.g. read, create, write, delete • Import the sample records from a spreadsheet	• Add a table mapping operation → role → condition • Provide the ACL script with comments to copy	• Provide a test checklist with the expected result for each role combination • Create a separate test user per role combination	• How might we drive access from the user's branch instead of hard-coded roles? • Use one group or role per branch